Assignment 3: Save the Campus Festival

The festival needs connected venues, a workable preparation schedule and reliable delivery routes. Your job is to implement a graph toolkit for these three graph models: undirected cables, directed dependencies and directed delivery routes.

Deadline: 18 October 2026, 23:59 (Asia/Qyzylorda, UTC+5). Work in your own copy of the GitHub template. Submit a committed ZIP snapshot in Moodle using the instructions at the end.

What you will learn

Implement and explain DSU, Kruskal, Prim, Kosaraju SCC, DFS topological ordering and DAG shortest paths. Validate witnesses, handle ties and edge cases, reconstruct routes and justify complexity. You should already know adjacency lists, DFS, sorting, priority queues and 64-bit arithmetic.

Program interface

One JSON object per input line, one JSON result object per line. One task per request. No prompts or debug output on stdout. Independent requests reset all task state. Provided adapters handle input/output. Inputs satisfy the published domain; malformed JSON is not graded.

UTF-8 JSON lines. Task names are case-sensitive.

All vertices and edges use zero-based indices. An edge ID is its position in the input edge array. All costs, distances, weights and counts use signed 64-bit integers; null denotes unreachable distance.

Task 1 - Connectivity / DSU

Input: {task:dsu,n,operations:[[u,v],...]}

Output: {states:[[components,largest],...]}

Limits: 0 <= n <= 100000; 0 <= q <= 250000 operations. If n=0, operations must be empty. Self-union and repeated union are valid.

Implement find and union, path compression and union by size or rank. Initially n singleton components; largest=0 for n=0. Return a state after every operation. A union within one component does not change the count or size.

Complexity: O((n+q) alpha(n)) amortized time, O(n+q) memory including output.

Example input:

```
{"task": "dsu", "n": 4, "operations": [[0, 1], [1, 2], [0, 2], [3, 3]]}
```

One accepted output:

```
{"states": [[3, 2], [2, 3], [2, 3], [2, 3]]}
```

Task 2 - Budget Network / Kruskal

Input: {task:kruskal,n,edges:[[u,v,weight],...]}

Output: {cost,edge_ids:[...],components}

Limits: 0 <= n <= 100000; 0 <= m <= 250000; -1000000000 <= weight <= 1000000000. Parallel edges and self-loops allowed.

Implement Kruskal using DSU. Return a minimum spanning forest, including all connected components and isolated vertices; components=0 for n=0. cost is the total forest cost. Return input edge IDs, with no repetitions. Self-loops are excluded. Any optimal forest is accepted. Negative weights represent discounts.

Complexity: O(n+m log(m+1)) time, O(n+m) memory.

Example input:

```
{"task": "kruskal", "n": 4, "edges": [[0, 1, 4], [1, 2, 2], [0, 2, 3]]}
```

One accepted output:

```
{"cost": 5, "edge_ids": [1, 2], "components": 2}
```

Venue 3 is isolated. The best available forest costs 5; it does not connect every venue.

Task 2 - Budget Network / Prim

Independently build the same minimum-cost forest by expanding one component at a time.

Input: {task:prim,n,edges:[[u,v,weight],...]}

Output: {cost,edge_ids:[...],components}

Limits: Same as Kruskal.

Implement Prim with adjacency lists and a priority queue. Restart at every unvisited component. Use the same forest contract as Kruskal. Costs and feasibility must agree, but selected edge IDs may differ. Compare algorithms on three input sizes in ANALYSIS.md.

Complexity: O(n+m log(m+1)) for a lazy heap; O((n+m) log(n+1)) for an indexed heap; O(n+m) memory.

Example input:

```
{"task": "prim", "n": 3, "edges": [[0, 1, 1], [1, 2, 1], [0, 2, 1]]}
```

One accepted output:

```
{"cost": 2, "edge_ids": [0, 2], "components": 1}
```

Several answers are optimal. The validator checks the forest rather than matching one reference edge list.

Task 3 - Circular Dependencies / SCC

Input: {task:scc,n,edges:[[u,v],...]}

Output: {component:[label per vertex],condensation:[[from_label,to_label],...]}

Limits: 0 <= n <= 100000; 0 <= m <= 250000. Directed edges; duplicates and self-loops allowed.

Implement Kosaraju with two DFS passes and a transposed graph. Labels are nonnegative integers; their values and order are unrestricted. Equal labels must mean exactly one SCC. Condensation contains each cross-component pair once and no internal edges. Handle isolated vertices and deep graphs. Condensation is a DAG, but internal dependency cycles still need a real scheduling decision.

Complexity: O(n+m) expected time with a hash set for edge deduplication; O(n+m) memory.

Example input:

```
{"task": "scc", "n": 4, "edges": [[0, 1], [1, 0], [1, 2], [0, 2], [2, 3]]}
```

One accepted output:

```
{"component": [5, 5, 7, 9], "condensation": [[5, 7], [7, 9]]}
```

Vertices 0 and 1 form one SCC. Two original edges become one condensation edge.

Task 4 - Festival Schedule / Topological Ordering

Input: {task:topo,n,edges:[[u,v],...]}

Output: {acyclic:boolean,order:[...]}

Limits: Same directed graph domain as SCC. Edge u->v means u must finish before v starts.

Implement DFS with unvisited/active/finished states and reverse postorder. A back edge to an active vertex indicates a cycle. For a cyclic graph return acyclic=false, order=[]. For a DAG list every vertex exactly once, respecting every edge. Any valid order is accepted. The empty graph has acyclic=true, order=[].

Complexity: O(n+m) time and memory, including explicit DFS stack.

Example input:

```
{"task": "topo", "n": 4, "edges": [[0, 2], [1, 2], [2, 3]]}
```

One accepted output:

```
{"acyclic": true, "order": [1, 0, 2, 3]}
```

Preparations 0 and 1 may be swapped; both must precede 2.

Task 5 - Delivery Routes / DAG Shortest Paths

Route supplies through a directed acyclic delivery graph. A downhill conveyor can give a negative edge cost.

Input: {task:dag,n,edges:[[u,v,weight],...],source}

Output: {acyclic:boolean,distance:[integer or null per vertex],parent_edge:[edge ID or -1 per vertex]}

Limits: 1 <= n <= 100000; 0 <= m <= 250000; -1000000000 <= weight <= 1000000000; 0 <= source < n. Directed graph; parallel edges and self-loops allowed. Cyclic input must be rejected as below.

Check acyclicity, then relax edges in topological order. On ANY directed cycle, including an unreachable cycle, return acyclic=false and both arrays empty. Otherwise source has distance=0 and parent_edge=-1. Unreachable vertices have distance=null and parent_edge=-1. Every other parent edge ends at that vertex and reconstructs an optimal source path. Provide a route reconstruction helper and show it in an example. Never relax from infinity. Multiple shortest routes are accepted; MST is unrelated to this objective.

Complexity: O(n+m) time and memory. Route reconstruction costs O(route length).

Example input:

```
{"task": "dag", "n": 5, "source": 0, "edges": [[0, 1, 3], [0, 2, 8], [1, 2,
  -7], [2, 3, 4]]}
```

One accepted output:

```
{"acyclic": true, "distance": [0, 3, -4, 0, null], "parent_edge": [-1, 0, 2, 3,
  -1]}
```

Route to 3 is 0->1->2->3 with cost 0. Venue 4 is unreachable. Negative DAG weights are valid.

Analysis and your own tests

Complete ANALYSIS.md in English or Russian: 3–5 sentences per algorithm about the invariant, steps, edge cases and complexity. Explain request reset, deep DFS, forest/condensation witnesses and route reconstruction. Compare Kruskal and Prim on the same graphs at three reproducible sizes. Record n, m, seeds or input files, runtime, commands, repetition count, median times and observations. Timing differences are for analysis, not a speed ranking.

Add your own inputs and expected/actual results in tests/student/. Cover ordinary inputs and relevant edge cases: empty/disconnected graphs, loops and parallel edges, ties, negative weights, cycles, unreachable vertices and repeated requests. These tests supplement the supplied public checks. State assistance and resources used, their contribution and your own verification, under the existing syllabus policy. You must understand your implementation; no additional AI restrictions apply. The syllabus permits AI for brainstorming, debugging and code suggestions with written acknowledgment; you remain responsible for correctness, originality and quality. Submitting AI-generated work without modification or acknowledgment is treated as plagiarism. The instructor may ask you to explain the submitted work.

Practical assessment: 50 points

Algorithm	Core	Edge cases	Witness	Scale	Total
DSU	4	2	0	2	8
Kruskal	4	0	2	1	7
Prim	4	0	2	1	7
SCC	5	0	3	2	10
Topological order	4	2	0	2	8
DAG shortest paths	5	0	3	2	10
Total					50

Core checks values/basic behaviour; edge checks published edge cases; witness checks valid optimal forests/routes and exact condensation; scale checks correct large results. Each group earns group points × passed checks / total checks, rounded half-up to 0.01. The practical sum is capped at 50. Correct core values can earn points even if the witness is invalid. A build failure prevents checks from earning points; a timeout loses the affected checks. A checker error gives no grade until investigated and rerun. Extra tests do not change group maxima or introduce unpublished requirements.

Official checks use the uploaded Moodle ZIP and a fixed instructor checker, not the workflow in your repository. Tests alone cannot prove the algorithm used: implement the specified methods and keep them clear enough to inspect. The analysis supports this review; it has no separate hidden scoring group. Standard collections, sorting and priority queues are allowed; external algorithm libraries cannot replace the required implementations.

Compilation limit: 60 seconds. Each requirement-group batch: 20 seconds, one CPU, 768 MiB container memory (Java heap up to 512 MiB), 128 processes and 8 MiB UTF-8 output. Representative large checks use up to 50,000 vertices, 100,000 DSU operations and 60,000 MST edges, within the full domains above. Linux GCC is used for official C++ checks.

A separate individual understanding quiz is worth 50 points. Its arrangements will be announced later. Existing practice PDFs are optional and ungraded. Practical work plus the quiz gives the assignment score out of 100. The syllabus weights remain unchanged.

From GitHub to Moodle

1. Open https://github.com/RZhaniya/daa-assignment-3-festival and choose Use this template → Create a new repository. Public or private copies are allowed; provide private-repository access to the instructor when requested. Install Git, Python 3.12+ and your runtime: JDK 17+, C++17 compiler or .NET SDK 8. Clone your own repository with Git, rather than downloading a ZIP.

2. Choose Java, Python, Cpp or CSharp in submission.json. Implement in that language's source/ folder; helper sources go at its top level. Keep the supplied JSON adapter working.

3. Run the commands below from the repository folder. Setup checks transport; TODO algorithms intentionally fail --check. Windows may use py -3 instead of python. All commands select the manifest language; --language overrides it for one run.

```
python tools/run.py --setup
python tools/run.py --check
python tools/run.py --input examples/dsu.jsonl
python tools/check_submission.py
```

4. Complete ANALYSIS.md, SUBMISSION.md and your own tests. Review local results and GitHub Actions → Public checks after a push. Enable Actions in your own copy if asked. Passing public checks is useful feedback, not a guarantee of full marks or authorship.

5. Commit and push the files. Confirm git status is clean. Generate the submission package with your numeric university student ID:

```
python tools/package_submission.py --student-id YOUR_NUMERIC_STUDENT_ID
```

6. Upload .submission/Assignment3_<ID>.zip to Assignment 3 in Moodle. Paste the three lines from .submission/Moodle_submission.txt into Online text: repository URL, full 40-character commit SHA and language. Save, click Submit assignment and confirm Submitted for grading before the deadline. A draft is not a completed submission.

The package includes your selected source, manifest, analysis, submission record, README and own tests. It excludes build output, other languages, public checking tools and practice PDFs. Packaging records a snapshot, not completion of all requirements. Keep the ZIP and Moodle confirmation. For changes before the deadline, commit and push again, repackage and replace the Moodle submission using its available controls. If locked, contact the instructor before the deadline. Later GitHub changes do not update Moodle's ZIP. The Moodle submission record determines timeliness; approved individual extensions take precedence.